Deep Research: Avatrada 57 Topic 042

Engineering Report: Stale and Spike Tick Filtering for Real-Time Data Streams

Authored By: Autonomous Technical Researcher **Date:** October 26, 2023
Subject: Deep-Dive on a Stateful Data Cleaning Filter for Real-Time WebSocket Ingestion

Executive Summary

This report provides a detailed engineering analysis of a stale and spike tick filtering mechanism designed for real-time data ingestion, as specified in the source documentation. The system is designed to discard data points that are either too old (stale) or represent statistically improbable price movements (spikes).

The core of the analysis focuses on the prompt's specific implementation requirements: a 500ms staleness check and a stateful 5% price deviation filter that requires confirmation from a subsequent tick. We deconstruct the logic, propose a robust implementation strategy using Python and a `deque` for efficient state management, and perform a critical analysis of potential failure modes, edge cases, and performance optimizations.

The proposed implementation successfully handles the confirmation logic by maintaining a `pending_spike` state, ensuring that transient noise is rejected while legitimate, rapid market shifts are eventually accepted. The analysis also contrasts the specified 5% rule with the more statistically robust 50-sigma approach mentioned in the source context, providing a path for future enhancement.

---

1. Technical Deconstruction

The system is a two-stage ingestion filter designed to clean a stream of financial ticks before they are processed by downstream systems. Each tick is assumed to have at least a `timestamp` and a `price`.

1.1. Mechanism 1: Stale Tick Filter

This is a stateless filter that acts as the first line of defense against network latency and data source delays.

- **Specification:** Discard any incoming tick if its timestamp is older than 500 milliseconds relative to the current time at the moment of ingestion.
- **Formula/Logic:** A tick is considered stale and is discarded if the following condition is true: `Ingestion_Time - Tick_Timestamp > 500ms`
- **Key Components:**
  - `Tick_Timestamp`: The timestamp embedded within the data packet, representing when the event (e.g., a trade) occurred at the source. This is typically a Unix timestamp with millisecond or microsecond precision.
  - `Ingestion_Time`: The timestamp generated by the filtering service's local clock at the moment it receives the tick.
  - `500ms`: A configurable latency threshold.

1.2. Mechanism 2: Stateful Spike Filter

This is a more complex, stateful filter designed to reject anomalous price movements that are likely data errors rather than legitimate market activity.

- **Specification:**
  1. **Detection:** Discard a tick if its price deviates by more than 5% from the price of the *last valid tick*.
  2. **Confirmation:** A detected spike is not immediately discarded. It is held in a pending state. If the *next* tick to arrive also represents a significant deviation (confirming a regime shift), the original spike is considered valid and is released. If the next tick returns to the

previous price level, the original spike is confirmed as noise and is permanently discarded.

- **Formula/Logic (Detection):** `| (Current_Price - Last_Valid_Price) / Last_Valid_Price | > 0.05`
- **Stateful Logic:** The filter must maintain state across multiple ticks:
  1. A history of the last N *valid* ticks.
  2. A temporary holding state for a `pending_spike`.

1.3. Data Structure: `deque`

The prompt specifies using a `deque` (double-ended queue) to manage the history of recent ticks.

- **Purpose:** To provide an efficient, sliding window of the most recent, validated data points.
- **Efficiency:** A `deque` with a maximum length (`maxlen`) provides O(1) time complexity for both appending new items (to the right) and removing old items (from the left), making it ideal for high-throughput stream processing.

2. Implementation Strategy

We will design a Python class, `TickFilter`, that encapsulates the state and logic. This class will process one tick at a time and yield valid ticks.

2.1. Core Data Structures and Class Design

We'll use Python's `collections.deque` and a simple `dataclass` for the tick structure.

```
import time
from collections import deque
from dataclasses import dataclass
from typing import Optional, Deque, Iterator

@dataclass
```

```
class Tick:
    """Represents a single data point from the stream."""
    timestamp: float  # Unix timestamp (e.g., from time.time())
    price: float
    symbol: str

class TickFilter:
    """
    A stateful filter for cleaning a real-time stream of ticks.
    """
    def __init__(self, history_size: int = 100, stale_ms: int = 500,
spike_threshold: float = 0.05):
        self.stale_threshold_s = stale_ms / 1000.0
        self.spike_threshold = spike_threshold

        # Deque to store the last N validated ticks
        self.valid_ticks: Deque[Tick] = deque(maxlen=history_size)

        # State for handling the confirmation logic
        self.pending_spike: Optional[Tick] = None

    def _is_stale(self, tick: Tick) -> bool:
        """Checks if a tick is older than the staleness threshold."""
        return time.time() - tick.timestamp > self.stale_threshold_s

    def _is_spike(self, current_price: float, last_price: float) -> bool:
        """Checks if a price deviates more than the spike threshold."""
        if last_price == 0: # Avoid division by zero
            return False
        return abs((current_price - last_price) / last_price) >
self.spike_threshold
```

2.2. Processing Algorithm

The core logic resides in a `process_tick` method. This method will be a generator, yielding zero, one, or two ticks per call, accommodating the spike confirmation logic.

```
 # Inside the TickFilter class

def process_tick(self, tick: Tick) -> Iterator[Tick]:
    """
    Processes a single incoming tick and yields valid ticks.
    """
    # 1. Stale Tick Filter
    if self._is_stale(tick):
        # print(f"Discarding stale tick for {tick.symbol}") # Optional: log this
        return

    # --- Stateful Spike Logic ---

    # Get the last known good price for comparison
    last_good_tick = self.valid_ticks[-1] if self.valid_ticks else None

    # 2. Handle a pending spike (Confirmation Stage)
    if self.pending_spike:
        # Compare the new tick against the last *valid* tick before the spike
        last_price_before_spike = self.valid_ticks[-1].price if
self.valid_ticks else self.pending_spike.price

        if self._is_spike(tick.price, last_price_before_spike):
            # CONFIRMED: The new tick also deviates. The market has moved.
            # Release the pending spike and the current tick.
            # print(f"Spike CONFIRMED for {self.pending_spike.symbol}")
            yield self.pending_spike
            self.valid_ticks.append(self.pending_spike)

            yield tick
            self.valid_ticks.append(tick)
        else:
            # REJECTED: The new tick returned to a normal level. The spike was
noise.
            # Discard the pending spike and process the current tick as normal.
            # print(f"Spike REJECTED for {self.pending_spike.symbol}")
            yield tick
            self.valid_ticks.append(tick)

        self.pending_spike = None # Reset state
```

```
        return

    # 3. Handle a new potential spike (Detection Stage)
    if last_good_tick and self._is_spike(tick.price, last_good_tick.price):
        # Detected a potential spike. Hold it for confirmation.
        # print(f"Detected potential spike for {tick.symbol}. Holding.")
        self.pending_spike = tick
        return

    # 4. Standard Case: Tick is valid and not a spike
    yield tick
    self.valid_ticks.append(tick)
```

2.3. Example Usage

```
 # Example WebSocket stream simulation
def stream_generator():
    base_time = time.time()
    yield Tick(timestamp=base_time - 0.1, price=100.0, symbol="BTC")
    yield Tick(timestamp=base_time - 0.05, price=101.0, symbol="BTC")
    # This tick is a >5% spike
    yield Tick(timestamp=base_time, price=107.0, symbol="BTC")
    # This tick confirms the spike
    yield Tick(timestamp=base_time + 0.1, price=108.0, symbol="BTC")
    yield Tick(timestamp=base_time + 0.2, price=108.5, symbol="BTC")
    # This is another spike
    yield Tick(timestamp=base_time + 0.3, price=95.0, symbol="BTC")
    # This tick rejects the spike by returning to the previous level
    yield Tick(timestamp=base_time + 0.4, price=109.0, symbol="BTC")

# --- Main execution ---
tick_filter = TickFilter()
print("Processing stream...")
for incoming_tick in stream_generator():
    print(f"-> In: {incoming_tick}")
    for valid_tick in tick_filter.process_tick(incoming_tick):
        print(f"<- Out: {valid_tick}\n")

# Expected Output:
```

```
# The 107.0 spike will be held, then released along with 108.0.
# The 95.0 spike will be held, then discarded when 109.0 arrives.
```

3. Critical Analysis

3.1. Potential Failure Modes & Edge Cases

- **Initialization:** The filter has no "last price" for the very first tick. The current implementation correctly handles this by requiring `last_good_tick` to exist before checking for a spike, implicitly accepting the first tick.
- **Stream Interruption with a Pending Spike:** If a spike is detected and the data stream then stops, the `pending_spike` will be held in memory indefinitely and will never be resolved.
    - **Mitigation:** Implement a timeout mechanism. If a new tick doesn't arrive within a certain window (e.g., 1-2 seconds) to resolve the pending spike, the filter should discard it to prevent state corruption.
- **Definition of "Confirmation":** The current logic confirms a spike if the subsequent tick is *also* a spike relative to the pre-spike price. An alternative interpretation could be that the subsequent tick is simply "close" to the spike's price. The current implementation is more robust against a single faulty tick followed by another, unrelated faulty tick.
- **Clock Skew:** The stale tick filter relies on synchronized clocks between the data source and the ingestion service. Significant clock drift could cause the filter to incorrectly discard valid ticks or accept stale ones.
    - **Mitigation:** Ensure all servers are synchronized to a reliable time source using NTP (Network Time Protocol).

3.2. Performance and Optimization

- **Computational Complexity:** The algorithm is highly efficient. Each `process_tick` call involves a few comparisons and arithmetic operations. The use of `deque` ensures that state management is O(1). The performance is dominated by I/O and is suitable for high-frequency streams on a single thread.

- **Python's GIL:** For extremely high-volume scenarios involving multiple symbols/streams processed in parallel, Python's Global Interpreter Lock (GIL) could become a bottleneck.
    - **Optimization:** For such cases, this logic could be ported to a language without a GIL (like Go, Rust, or C++) or parallelized using a multi-processing architecture where each process handles a subset of streams.
- **Memory Usage:** The memory footprint is determined by the `history_size` of the deque. For 100 ticks, this is negligible. For millions of ticks (e.g., for calculating long-term moving averages), memory should be monitored.

3.3. Alternative Approaches & Enhancements

- **50-Sigma Filter (from Source Context):** The 5% fixed threshold is simple but brittle. A more robust method is to use a statistical measure.
    - **Implementation:** Instead of a fixed percentage, calculate the rolling mean (μ) and standard deviation (σ) of price *changes* over the `valid_ticks` deque. A tick is an outlier if `|tick.price - last_tick.price| > μ + 50 * σ`.
    - **Trade-offs:** This is more computationally intensive as it requires recalculating μ and σ on each tick, but it adapts to changing market volatility. A 50-sigma threshold is extremely high and would only catch the most egregious errors. A more common value would be 3-5 sigma.
- **Volume Confirmation:** A price spike could be validated by looking at trading volume. A large price change on very low volume is more likely to be an error than one accompanied by high volume. This would require the `Tick` object to include volume data.
- **Multi-level Confirmation:** Instead of requiring just one subsequent tick for confirmation, the logic could be extended to require M of the next N ticks to confirm the new price level, making the filter more resilient to short-lived noise.